TRUE 贞实科技

AI 赋能控制

Topic 例程
使用手册

V1.5.1

2026.1

文档版本

版本号	修订日期
1.4	2025 年 07 月 15 日
1.5	2025 年 07 月 23 日
1.5.1	2026 年 01 月 09 日

目录

文档版本 II
1. 在 Linux 中使用 topic 4
2. topic 程序说明 5
附录 6

1. 在 Linux 中使用 topic

Linux 环境下打开\topic_demo\linux 文件夹，有 topic.so 、main.py、message.py、libprotobuf.so、libzmq.so、libprotobuf.so.32、libzmq.so.5 七个文件

其中 main.py 是订阅端 python 的例程

message.py 是订阅端能打印的参数结构体

topic.so 是 topic 通信 python 的依赖库，另外四个 so 文件则是 topic.pyd 的依赖库

依赖库可忽略，用户只需关注例程和参数结构体即可

终端进入到此目录中使用指令 python main.py 或 python3 main.py

```
z@z-virtual-machine:~/桌面/test/linux$ python3 sub.py
Subscribing1 to: tcp://localhost:19091
Subscribing2 to: tcp://localhost:19091
NodeOptions
        name: {test1}
        pub url: {}
        sub url: {tcp://localhost:19091}
        thread poll size: {1}
        pub queue length: {50}
        sub queue length: {1000}
init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://localhost:19091}
sub_monitor_thread_ started.
RTNode started successfully.
LoopRecv started...
NodeOptions
        name: {test2}
        pub url: {}
        sub url: {tcp://localhost:19091}
        thread poll size: {1}
        pub queue length: {50}
        sub queue length: {1000}
init
on_monitor_started
zmq_version 4.3.6
[SUB] connect: {tcp://localhost:19091}
NRTNode started successfully.
Creating subscription for topic: system_rtstate
Creating subscription for topic: system_nrtstate
sub_monitor_thread_ started.
LoopRecv started...
```

若报如下错误，则需将 libprotobuf.so.32 文件复制到根目录的 lib 文件夹中

```
z@z-virtual-machine:~/桌面/test/message x86/lib/linux/python$ python3 sub.py
Traceback (most recent call last):
  File "/home/z/桌面/test/message x86/lib/linux/python/sub.py", line 2, in <module>
    import topic
ImportError: libprotobuf.so.32: cannot open shared object file: No such file or directory
```

libzmq.so.5 缺失的错误相同处理

2. topic 程序说明

程序分为几个部分：配置、创建节点，订阅远程地址，订阅话题名称，回调函数打印话题信息。

创建 topic 节点，在回调函数中分别订阅"system_rtstate"及"system_nrtstate"主题来接收 消息。其中 system_rtstate 是实时接收，数据更新速度较快，system_nrtstate 是非实时接收，数据更新速度较慢。

程序中已经将所以接口信息全部打印，可以根据需求自行取用。

```python
# 1. 定义回调函数，处理接收到的 system_rtstate 消息
def on_system_rtstate_update(tt: topic.SystemRtState):…

# 定义回调函数，处理接收到的 system_nrtstate 消息
def on_system_nrtstate_update(tt: topic.SystemNrtState):…

# system_rtstate中常用信息-快速使用
def on_system_basicstate_update(tt: topic.SystemRtState):…


# 2. 配置节点选项
system_tpoic_options = topic.NodeOptions()
system_tpoic_options.node_name = 'test1'
system_tpoic_options.sub_url = 'tcp://192.168.2.153:19091'
safe_print("Subscribing1 to:", system_tpoic_options.sub_url)


# 3. 创建节点并启动
system_topic_node = topic.Node(system_tpoic_options)
if not system_topic_node.Start():
    safe_print("Failed to start node.")
else:
    safe_print("RTNode started successfully.")


# 4. 创建订阅
sub1 = system_topic_node.CreateSubscriptionRT("system_rtstate", on_system_rtstate_update)
sub2 = system_topic_node.CreateSubscriptionNRT("system_nrtstate", on_system_nrtstate_update)
sub3 = system_topic_node.CreateSubscriptionRT("system_rtstate", on_system_basicstate_update)


# 5. 保持程序运行，持续接收消息
try:
    while True:
        time.sleep(0.01)  # 防止主线程退出
```

```python
# 1. 定义回调函数，处理接收到的 system_rtstate 消息
def on_system_rtstate_update(tt: topic.SystemRtState):
    print("\n" * 1)
    print("Received system rtstate update")
    print("Debug: on_system_rtstate_update called")  # 调试日志
    parm_rt = message.SystemStateData()
    message.display_rt(tt, parm_rt)

    print("********************* System State Update ******************************")

    print(f"""
            1: header_timestamp: {parm_rt.header_timestamp}
            2: header_frame_id: {parm_rt.header_frame_id}
            3: system_running_state: {parm_rt.system_running_state}
            4: controller_name: {parm_rt.controller.controller_name}
            5: control_cycle: {parm_rt.controller.control_cycle}
            6: global_count: {parm_rt.controller.global_count}
            7: master_info: {parm_rt.controller.master_info}
            8: is_link_up: {parm_rt.controller.is_link_up}
    """)

    b = 1
    for model_info in parm_rt.models:
        print("********************* Models and Joints Information num: {b} **********
        print(f"model num:{b} 1: Model Name: {model_info.model_name}")
        print(f"model num:{b} 2: Model Type: {model_info.model_type}")
        b = b + 1
```

注：std::count 的打印方式没有线程保护，直接观察打印信息会存在顺序错乱问题属于正常现象，现有打印方式仅提供各接口以及调用方法，可根据需求进行选取调用。

附录

法律声明

1. 版权声明

© 2025 贞实科技版权所有。未经书面授权，禁止以任何形式复制、传播本文件内容。

2. 免责条款

使用者应自行承担因以下情况导致的损失风险：

- 未按指引操作造成的系统配置错误
- 擅自修改动态库文件引发的程序崩溃（如 message.dll 等库非授权替换）
- 开发环境版本不匹配导致的编译失败（如 CMake 2.8 与 VS2022 兼容问题）

3.数据隐私条款（Data Privacy）

产品运行过程中产生的日志文件，仅包含设备状态信息，不会收集用户个人信息。